EXPERIMENT 11

AIM: MINI PROJECT

Submit the Project Report which will consist of documentation related to different phases of Software Development Life Cycle:

- Title of the Project, Abstract, Introduction
- Software Requirement Specification
- Conceptual Design using ER features, Relational Model in appropriate Normalize form
- Graphical User Interface, Source Code
- Testing document
- Conclusion

Develop application considering:

- **Front End:** React / JavaScript / Tailwind CSS / Recharts
- **Backend :** Python (FastAPI) / SQLite / Scikit-learn / Groq LLM



NetGuard AI Cyber Defense Architecture

A Mini Project Report

on

NetGuard AI: Network Traffic Anomaly Detection System

by

Candidate (SE-Computer)

Under the guidance of

Prof. Harsha Jain



Department of Computer Engineering

K J's Educational Institute

K J College of Engineering & Management Research, Pune

SAVITRIBAI PHULE PUNE UNIVERSITY

2025-2026

K J's Educational Institute

K J College of Engineering & Management Research

(Accredited by NAAC)

Affiliated to Savitribai Phule Pune University (SPPU), Approved by Govt. of Maharashtra, Recognized by AICTE.

CERTIFICATE

This is to certify that,

Candidate (SE-Computer)

of class **S.E Computer** has successfully completed the mini project work on **"NetGuard AI: Network Traffic Anomaly Detection System"** at **K J College of Engineering & Management Research, Pune** in the partial fulfilment of the Graduate Degree course in S.E at the Department of Computer Engineering in the academic Year **2025-2026 Semester - IV** as prescribed by the Savitribai Phule Pune University.

Prof. Harsha Jain
Project Guide

Dr. Nikita Kulkarni
Head of Department

Acknowledgements

With a deep sense of gratitude, we would like to thank all the people who have guided and supported us throughout the development of our project **“NetGuard AI: Network Traffic Anomaly Detection System”**. Their valuable suggestions and encouragement helped us to complete this work successfully.

It is our proud privilege to express our sincere thanks to **Dr. Suhas S. Khot**, Principal of K J College of Engineering & Management Research, Pune, for providing us the opportunity and necessary facilities to carry out this project. We are highly thankful to **Dr. Nikita Kulkarni**, Head of the Computer Engineering Department, for her constant encouragement and continuous guidance during the project work.

We express our special gratitude to our project guide, **Prof. Harsha Jain**, for providing us with expert advice, motivation, and support at every stage of project development. Her knowledge and experience helped us understand the practical implementation of Machine Learning and Database Management System concepts in real-world applications.

We would also like to thank all the faculty members, friends, and classmates for their support and cooperation, which helped us overcome various challenges faced during the completion of this project. Finally, we thank everyone who directly or indirectly contributed to the successful completion of NetGuard AI.

Contents

Sr.No.	Topic	Page No.
	Acknowledgement	i
	Content	ii
	Abstract	iii
Chapter 1	Introduction	1
Chapter 2	Literature Review	3
Chapter 3	Software and Hardware Requirement Specification	6
	3.1 Hardware Requirement	
	3.2 Software Requirement	
Chapter 4	Entity Relationship Diagram	8
Chapter 5	Tables/Collection	9
Chapter 6	Result	11
Chapter 7	Conclusion	16
	References	18

Abstract

The **NetGuard AI: Network Traffic Anomaly Detection System** is a relational database-driven, machine learning-powered application designed to detect malicious intrusions and anomalous traffic vectors efficiently. The system is developed using FastAPI (Python) and structured relational database techniques (SQLite via SQLAlchemy 2.0) and follows normalization principles to eliminate redundancy and maintain high data consistency.

It supports multiple modules such as raw dataset upload and validation, supervised Random Forest classification (88.0% Test Accuracy, 0.98 ROC-AUC, 0.91% False Positive Rate), unsupervised Isolation Forest anomaly detection, automated security alert triage, and a Groq-powered AI Cybersecurity Assistant grounded in SQLite telemetry.

It eliminates the drawbacks of traditional manual log inspection and static firewalls such as high false positives, lack of contextual explainability, and time consumption. The system ensures data integrity, security, and fast retrieval through well-designed relational tables and RESTful endpoints.

This project highlights the practical implementation of DBMS concepts like ER modeling, relational schema design, and query processing in a real-world cybersecurity application.

Chapter 1: Introduction

1.1 Introduction:

In traditional networks, traffic inspection and threat management are done manually or using rigid static rules, which is time-consuming and error-prone. Managing high-volume flow records, identifying subtle DDoS attacks, and responding to port scans becomes difficult as data increases.

This system uses Database Management System (DBMS) concepts to maintain records such as destination ports, flow duration, packet counts, anomaly scores, classification verdicts, and security incident alerts. It provides a centralized platform where all network flow information is stored in a structured format, reducing redundancy and ensuring consistency.

The NetGuard AI System is developed to:

- Store and manage large amounts of network flow capture data
- Provide quick access to anomaly predictions and telemetry
- Reduce manual effort and error in incident triage
- Improve overall efficiency of Security Operations Centers (SOCs)

1.2 Problem Statement:

Managing network traffic anomalies manually or using separate disconnected tools leads to inefficiency, data redundancy, and errors. Enterprises need to handle large amounts of flow information such as Destination Ports, Flow Durations, Forward and Backward packet counts, and attack signatures. Traditional methods make it difficult to store, update, and retrieve this data accurately in real time.

The NetGuard AI System is designed to overcome these problems by using a centralized database and ML pipeline. It helps in organizing and managing all traffic data efficiently. The system allows proper storage of uploaded datasets, flow predictions, severity alerts, and chat transcripts.

The main objective is to reduce manual work, avoid duplication of data, and ensure data consistency and security. The system also provides easy access to authorized analysts, enabling smooth triage of security incidents.

1.3 Framework of the Proposed Work in Project:

1. Needs Assessment and Requirement Gathering:

- Identify the needs of SOC analysts, administrators, and security teams.
- Study the problems faced in manual packet inspection and static rule firewalls.
- Collect benchmark network flow dataset records (CIC-IDS2017).

2. System Design and Architecture:

- Design database structure for storing datasets, detection runs, predictions, and alerts.
- Plan user interface for easy access and management of network threat information.
- Define relationships between entities such as datasets, runs, predictions, and alerts.

3. Development and Coding:

- Develop the system using suitable programming languages (Python, JavaScript) and DBMS tools (SQLite, SQLAlchemy).
- Implement core features such as dataset upload, Random Forest inference, and alert triage.
- Apply validation techniques to ensure accurate flow feature schema entry.

4. Testing and Quality Assurance:

- Test the system for accuracy, reliability, and performance using automated Pytest suites.
- Ensure proper functioning of modules like traffic analytics, anomaly detection, and AI assistant.

5. User Training and Documentation:

- Prepare user manuals for security analysts and administrators.
- Provide instructions for using system features and querying the AI assistant efficiently.

6. Launch and Deployment:

- Deploy the system for use by security analysts and students on cloud platforms (Render).
- Collect feedback for improvements.

7. Scalability and Enhancement:

- Improve system performance to handle large amounts of network traffic.
- Add advanced features such as live PCAP packet sniffing and automated firewall rule generation.
- Upgrade the system based on user feedback and future security requirements.

Chapter 2: Literature Review

2.1 Introduction:

NetGuard AI is a database-driven, machine learning application designed to manage and detect network anomalies in an efficient manner. With the increasing number of network connections, encrypted protocols, and automated attacks, it becomes difficult to handle all operations manually. Therefore, a computerized system is required to store, process, and retrieve threat information quickly and accurately.

This system uses Database Management System (DBMS) concepts to maintain records such as uploaded dataset files, detection runs, predictions, alerts, model metrics, and chat sessions. It provides a centralized platform where all information is stored in a structured format, reducing redundancy and ensuring data consistency.

The NetGuard AI system helps different users such as SOC analysts, administrators, and incident handlers to access relevant information easily. It reduces alert fatigue, saves time, and increases overall efficiency in cybersecurity operations.

2.2 Existing Methodologies:

1. Manual Analysis Method:

• Traditionally, security teams analyze packet capture files manually using Wireshark, which is time-consuming and prone to errors. This method lacks proper organization and makes it difficult to detect subtle distributed attacks in real time.

2. Database Management Systems (DBMS):

• DBMS technology has been widely used to store, organize, and manage large amounts of structured data. It helps in efficient data retrieval, updating records, and maintaining data consistency across incidents.

3. User-Centered Design Approach:

• Modern security systems follow user-centered design principles to ensure easy navigation and understanding. Interfaces are designed to be simple and user-friendly for SOC analysts with dark themes and visual charts.

4. Web-Based Application Systems:

• Modern systems are developed as web applications, allowing users to access information anytime and anywhere. These systems improve accessibility and provide centralized data storage.

5. Data Analysis Techniques:

• Machine Learning and data analysis methods are used to identify patterns and trends in network flows. Feature extraction and tree ensembles help in determining anomalous packet sizes and flow duration signatures.

6. Secure Data Storage Methods:

• Security techniques are used to protect network and forensic data from unauthorized access. Authentication, parameterized queries, and environment variable isolation help maintain data integrity and reliability.

2.3 Proposed Methodology:

1. Requirement Analysis:

- Identify the needs of SOC analysts, security engineers, and administrators.
- Analyze problems in existing manual tools such as data redundancy, alert fatigue, and errors.
- Gather requirements for modules like dataset upload, ML detection, alert triage, and AI advisory.

2. System Design and Database Structure:

- Design database tables such as: uploaded_datasets, detection_runs, traffic_predictions, security_alerts, model_evaluations, chat_sessions, chat_messages.
- Define relationships using ER diagrams (e.g., dataset-run, run-prediction, prediction-alert).
- Apply normalization (3NF) to ensure efficient data storage and avoid redundancy.

3. Development and Implementation:

Develop the system using modern web technologies (React, Tailwind CSS, JavaScript) and DBMS (SQLite via FastAPI). Implement modules such as:

1. Dataset upload, schema validation, and 20-row live preview
2. Random Forest and Isolation Forest inference engine
3. Interactive Recharts analytics and threat radar video feed
4. Security alert triage and analyst note management
5. Groq LLM Assistant grounded in SQLite telemetry

4. Testing and Validation:

- Conduct unit testing and system testing for all API routes and ML predictors.
- Validate data inputs to maintain accuracy and consistency across Excel and CSV files.
- Ensure proper functioning of operations like data insertion, updating, and retrieval.

5. User Guidance and Documentation:

Prepare user manuals and system documentation explaining features and usage. Provide guidelines for:

- Uploading and analyzing network captures (.xlsx/.csv)
- Managing security alerts and updating investigation status
- Querying the Groq AI Cybersecurity Assistant for containment guidance
- Exporting prediction CSV reports

6. Deployment and Data Entry:

- Deploy the full-stack unified system for online access using Render cloud.
- Load and benchmark the CIC-IDS2017 dataset into the relational database.

Chapter 3 : Software Requirement Specification

3.1 Hardware Requirements:

- Computer/Laptop with Intel i3 or higher (i5/i7 recommended)
- Minimum 4 GB RAM (8 GB recommended for ML inference)
- 256 GB storage / SSD
- Keyboard, mouse, display

3.2 Software Requirements:

- Operating System: Windows 10/11 or Linux
- Frontend: React 18, Vite, Tailwind CSS, JavaScript
- Backend: Python 3.12, FastAPI, Uvicorn
- Database: SQLite 3 via SQLAlchemy 2.0 ORM
- Machine Learning: Scikit-learn, Pandas, NumPy, Joblib
- AI LLM Core: Groq Python SDK (openai/gpt-oss-120b)
- Editor: VS Code / Notepad++
- Browser: Google Chrome / Edge

The above software helps in developing a user-friendly web application and managing data efficiently using DBMS and ML concepts.

Chapter 4 : Entity Relationship Diagram

The Entity-Relationship model defines the conceptual data structure of NetGuard AI. It illustrates the entities, attributes, primary keys, foreign keys, and relationships governing datasets, detection runs, predictions, alerts, evaluations, and AI chat sessions.

Entity	Primary Key	Foreign Key	Attributes	Cardinality
UploadedDataset	id (UUID)	-	filename, file_size, row_count, columns_json	1 : N with DetectionRun
DetectionRun	id (UUID)	dataset_id	model_name, total_records, normal_count, anomaly_pct	1 : N with TrafficPrediction
TrafficPrediction	id (Integer)	run_id	destination_port, flow_duration, packets, label, severity	1 : 1 with SecurityAlert
SecurityAlert	id (UUID)	run_id, prediction_id	alert_title, attack_category, severity, status, notes	Child of DetectionRun
ModelEvaluation	id (Integer)	-	model_name, accuracy, precision, recall, f1, roc_auc	Independent Metrics Log
ChatSession	id (UUID)	-	title, active_run_id, created_at, updated_at	1 : N with ChatMessage
ChatMessage	id (Integer)	session_id	role (user/assistant), content, context_run_id	Child of ChatSession

Relational Constraints and Multiplicities:

- **UploadedDataset to DetectionRun (1 to Many):** A single dataset can be evaluated repeatedly across Random Forest and Isolation Forest.
- **DetectionRun to TrafficPrediction (1 to Many):** A detection run generates individual prediction records for each network flow in the dataset.
- **TrafficPrediction to SecurityAlert (1 to 1 optional):** Every anomalous prediction is automatically transformed into an actionable incident alert.
- **ChatSession to ChatMessage (1 to Many):** A session maintains full multi-turn conversational advisory history with Groq.

Chapter 5 : Tables/Collections

5.1 Database Creation

CREATE DATABASE Statement

The CREATE DATABASE statement / SQLite engine initialization is used to create a new database.

Syntax:

```
CREATE DATABASE netguard_db;
```

Database: Database netguard.db (SQLite via SQLAlchemy)

5.2 Tables

Table: uploaded_datasets

```
CREATE TABLE uploaded_datasets (
   dept_id INT(10) PRIMARY KEY, -- id VARCHAR(36)
   filename VARCHAR(255) NOT NULL,
   file_size INT(10) NOT NULL,
   row_count INT(10) NOT NULL,
   column_count INT(10) NOT NULL,
   columns_json TEXT NOT NULL,
   preview_json TEXT NOT NULL,
   created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  );
```

Table: detection_runs

```
CREATE TABLE detection_runs (
   run_id VARCHAR(36) PRIMARY KEY,
   dataset_id VARCHAR(36) REFERENCES uploaded_datasets(id),
   dataset_name VARCHAR(255) NOT NULL,
   model_name VARCHAR(100) NOT NULL,
   total_records INT(10) NOT NULL,
   normal_count INT(10) NOT NULL,
   anomaly_count INT(10) NOT NULL,
   anomaly_percentage FLOAT NOT NULL,
   duration_ms FLOAT DEFAULT 0.0,
   status VARCHAR(50) DEFAULT 'completed',
   created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  );
```

Table: traffic_predictions

```
CREATE TABLE traffic_predictions (
   prediction_id INT(10) PRIMARY KEY AUTOINCREMENT,
   run_id VARCHAR(36) REFERENCES detection_runs(id),
   record_index INT(10) NOT NULL,
   destination_port INT(10) NOT NULL,
   flow_duration FLOAT NOT NULL,
   total_fwd_packets INT(10) NOT NULL,
   total_backward_packets INT(10) NOT NULL,
   total_fwd_len FLOAT NOT NULL,
   total_bwd_len FLOAT NOT NULL,
   true_label VARCHAR(100),
   predicted_label VARCHAR(100) NOT NULL,
   is_anomaly BOOLEAN NOT NULL,
   anomaly_score FLOAT NOT NULL,
   confidence FLOAT NOT NULL,
   severity VARCHAR(50) DEFAULT 'low'
  );
```

Table: security_alerts

```
CREATE TABLE security_alerts (
   alert_id VARCHAR(36) PRIMARY KEY,
   run_id VARCHAR(36) REFERENCES detection_runs(id),
   prediction_id INT(10) REFERENCES traffic_predictions(id),
   record_index INT(10) NOT NULL,
   alert_title VARCHAR(255) NOT NULL,
   attack_category VARCHAR(100) NOT NULL,
   severity VARCHAR(50) NOT NULL, -- critical, high, medium, low
   destination_port INT(10) NOT NULL,
   evidence_summary TEXT NOT NULL,
   investigation_status VARCHAR(50) DEFAULT 'New', -- New, Investigating, Resolved
   notes TEXT,
   timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
  );
```

Table: model_evaluations

```
CREATE TABLE model_evaluations (
   eval_id INT(10) PRIMARY KEY AUTOINCREMENT,
   model_name VARCHAR(100) NOT NULL,
   model_type VARCHAR(50) NOT NULL, -- Supervised, Unsupervised
   accuracy FLOAT NOT NULL,
   precision_score FLOAT NOT NULL,
   recall_score FLOAT NOT NULL,
   f1_score FLOAT NOT NULL,
   roc_auc FLOAT,
   fpr FLOAT,
   confusion_matrix_json TEXT NOT NULL,
   trained_at DATETIME DEFAULT CURRENT_TIMESTAMP
  );
```

Table: chat_sessions

```
CREATE TABLE chat_sessions (
   session_id VARCHAR(36) PRIMARY KEY,
   title VARCHAR(255) DEFAULT 'SOC Security Assessment',
   active_run_id VARCHAR(36),
   created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
   updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
  );
```

Table: chat_messages

```
CREATE TABLE chat_messages (
   message_id INT(10) PRIMARY KEY AUTOINCREMENT,
   session_id VARCHAR(36) REFERENCES chat_sessions(id),
   role VARCHAR(50) NOT NULL, -- user, assistant
   content TEXT NOT NULL,
   context_run_id VARCHAR(36),
   created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  );
```

Chapter 6 : Results

Fig 6.1 : Home Page (Command Overview & Live Threat Radar)

Fig. Home Page includes: The UI layout includes a top navigation header with system title, active dataset switcher, system health indicators, and a responsive collapsible sidebar menu for 9 different operational modules.

It provides a comprehensive dashboard with summary KPI cards: Total Scanned Records (1,000), Normal Traffic (55.1%), Anomalies Flagged (44.9%), and Active Security Alerts.

It features an embedded **Live Cyber Threat Radar Video Feed** with real-time HUD telemetry simulation, radar sweep overlay, and audio/video controls. Beside the radar, an interactive Recharts donut chart displays the Traffic Verdict Ratio, and a Top Targeted Destination Ports card details ingress attack concentrations on Port 21 (FTP), Port 22 (SSH), and Port 8080 (DDoS).

Fig 6.2 : Dataset Upload & Live 20-Row Preview Page

The Dataset Upload Page is used to enter and store new network flow captures into the system database. It provides a drag-and-drop zone and file browser supporting .xlsx, .xls, and .csv files.

The page automatically validates incoming columns against 10 canonical flow features (Destination Port, Flow Duration, Packet counts, Packet lengths). It renders an interactive 20-row live preview table before inference, features a model selector (Random Forest vs Isolation Forest), and provides a 1-click button to analyze the benchmark CIC-IDS2017 dataset.

Fig 6.3 : Traffic Flow & Threat Vector Analytics Page

The Analytics module visualizes multi-dimensional threat vectors using interactive Recharts components:

- **Destination Port Breakdown:** Grouped bar chart comparing normal vs anomalous packet volumes across ports 21, 22, 53, 80, 8080.
- **Flow Duration Latency Histogram:** Binned distribution from sub-millisecond flows (<1ms) to multi-second persistent streams (>10s).
- **Anomaly Confidence Spectrum:** Probability bins from 0.0 to 1.0 reflecting classifier certainty.
- **Attack Classification Breakdown:** Color-coded pie chart detailing percentage shares for DoS Hulk, DDoS, PortScan, and Patator.

Fig 6.4 : Anomaly Detection & Forensic Record Inspection Page

The Anomaly Detection page presents a paginated, searchable, and filterable table of individual flow predictions calculated by the model. Analysts can filter by verdict (All/Normal/Anomalous), severity (Critical/High/Medium/Low), or attack signature.

Clicking any record opens the **Forensic Record Modal**, exposing all 10 network flow parameters with numerical units (Destination Port, Flow Duration in us, Packet lengths). It includes a 1-click button: *"Ask AI Assistant to Analyze This Record"* which instantly transfers the flow parameters to the Groq chatbot for root-cause diagnosis. An Export CSV button streams predictions directly.

Fig 6.5 : Security Alerts Incident Triage Board

The Security Alerts page organizes incidents generated from anomalous flows into three lifecycle tabs: **New**, **Investigating**, and **Resolved**.

Severity is automatically assigned using transparent rules: Critical for volumetric DDoS and sensitive ports, High for SSH/FTP brute force, and Medium for port scans. Analysts can update the investigation status with a single dropdown click and attach persistent notes (e.g., "Firewall rule 402 applied to block Port 8080 ingress") saved directly to SQLite.

Fig 6.6 : Groq-Powered AI Cybersecurity Assistant Page

The AI Cybersecurity Assistant provides conversational reasoning powered by Groq cloud LPU inference (openai/gpt-oss-120b). Unlike generic chatbots that hallucinate, NetGuard AI retrieves verified metrics from SQLite (exact flow counts, anomaly rates, targeted ports) and injects them into system prompts.

The interface features message bubbles, user/assistant avatars, suggested query chips ("Why did the model flag port 21 / 22?", "Explain precision and recall metrics"), session history persistence, and defensive containment recommendations.

Fig 6.7 : Machine Learning Performance & Benchmarks Page

The Model Performance page displays verified statistical benchmarks evaluated on a held-out 20% test partition of the CIC-IDS2017 dataset:

- **Test Accuracy:** 88.0% across all multi-class signatures
- **Weighted Precision:** 87.1% (High positive predictive value)
- **Weighted Recall:** 88.0% (Comprehensive detection coverage)
- **F1-Score:** 87.1% (Harmonic mean of precision and recall)
- **ROC-AUC Score:** 0.980 (Exceptional class separability)
- **False Positive Rate (FPR):** 0.91% (Prevents alert fatigue)

The page includes an interactive **Confusion Matrix Heatmap** displaying actual versus predicted counts, a per-signature classification report table, a Gini Feature Importance ranking chart, and a 1-click Retrain Models button.

Fig 6.8 : Settings & System Diagnostics Page

The Settings page provides detailed information about NetGuard AI and its operational status. It helps users understand the system, backend diagnostics, and security configurations.

This page explains how the system helps reduce manual work, improves accuracy, and provides quick access to threat telemetry. It highlights key features of the system such as database storage, model caching, retrieval, updating, and report generation.

It includes:

- **Project Objective:** To detect and manage network traffic anomalies efficiently using DBMS and ML
- **System Description:** Handles dataset captures, runs, predictions, alerts, and chat transcripts
- **Key Features:** Data storage, vectorized inference, live threat radar, Recharts analytics, Groq AI assistant
- **Advantages:** Reduces manual work, improves accuracy, saves time, eliminates hallucinations
- **Technology Used:** React 18, Vite, Tailwind CSS (Frontend), Python FastAPI, SQLite, Scikit-learn, Groq (Backend)
- **Security Architecture:** Zero client-side API key exposure, parameterized queries, local SQLite data sovereignty
- **Project Details:** Information about developers, project guide (Prof. Harsha Jain), and institution

Chapter 7:Conclusion

7.1 Advantages

- Reduces manual paperwork and packet inspection effort
- Fast and easy access to flow prediction data (sub-50ms inference)
- Improves accuracy and reduces errors (88% Accuracy, 0.98 ROC-AUC)
- Centralized data management with SQLite relational schema
- Saves time in searching and updating security incident records
- User-friendly dark-navy SOC interface with live video telemetry

7.2 Disadvantages

- Initial setup takes time and effort
- Requires basic technical knowledge of network flow features
- Dependent on system and database availability
- Risk of data loss if database is not properly backed up

7.3 Applications

- Used in universities, colleges, and academic networks
- Helps SOC administrators manage security records
- Useful for incident response teams to triage DDoS and brute force
- Helps students and researchers study machine learning intrusion detection
- Can be used in data centers and enterprise network monitoring

7.4 Future Scope

- Development of mobile application for on-the-go alerts
- Integration with cloud databases (PostgreSQL/MongoDB)
- Addition of live PCAP packet sniffing via Scapy
- Online portal with multi-tenant role-based login system
- Automated firewall rule generation (iptables, AWS Security Groups)

7.5 Conclusion

The **NetGuard AI: Network Traffic Anomaly Detection System** is an efficient and reliable solution for managing and securing academic and enterprise network data using DBMS and Machine Learning. It simplifies the process of storing, organizing, and retrieving information related to network captures, threat signatures, predictions, and alerts.

The system reduces manual effort, improves accuracy, and saves time. Overall, it enhances the performance of educational and corporate institutions and provides a structured way to handle large amounts of network data effectively.

References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S., Database System Concepts, McGraw-Hill Education, 7th Edition.

2. Elmasri, R., & Navathe, S. B., Fundamentals of Database Systems, Pearson Education, 7th Edition.

3. Coronel, C., & Morris, S., Database Systems: Design, Implementation, and Management, Cengage Learning.

4. Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A., "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization (CIC-IDS2017)", Proceedings of ICISSP.

5. Pedregosa, F., et al., "Scikit-learn: Machine Learning in Python", Journal of Machine Learning Research (JMLR).

6. Breiman, L., "Random Forests", Machine Learning Journal.

7. Tiangolo, S., "FastAPI: Modern, High-Performance Web Framework for Python", Available at: https://fastapi.tiangolo.com/

8. Groq Inc., "Groq LPU Inference Documentation and Python SDK", Available at: https://console.groq.com/docs

9. W3Schools, "SQL Tutorial and Database Concepts", Available at: https://www.w3schools.com/sql/

10. GeeksforGeeks, "DBMS and SQL Tutorials", Available at: https://www.geeksforgeeks.org/dbms/

11. Mozilla Developer Network (MDN), "HTML5 and CSS Documentation", Available at: https://developer.mozilla.org/

12. TutorialsPoint, "Database Management System Tutorial", Available at: https://www.tutorialspoint.com/dbms/

13. Personal implementation and testing using SQLite, FastAPI, Scikit-learn, React, and Groq LLM interface.